AUTOMATA LEARNING LAB · O'REILLY LIVE TRAINING

Local Agents with *Hermes*

A persistent personal agent, explained against the Claude Code you already run.

2h 30m · links on these slides are clickable

AGENDA

What we'll cover

1. **The landscape** — models, harnesses, and where Hermes sits · 30m
2. **Your first agent** — install, context files, tools, evals · 30m
3. **Memory & skills** — the part only Hermes defaults to · 30m
4. **MCP, deploy, reach** — extend it, ship it, secure it · 25m
5. **Choosing** — Hermes vs Claude Code vs OpenClaw · 5m

BLOCK 1

The local AI agent *landscape*

No installs yet. Mental map first.

"The personal in personal agent is not the model... it's the agent's memory of your conversations and the skills you've built with it."

— Karan Malhotra, co-founder, Nous Research

No weights change — anywhere

"Self-improves through use, not retraining" means no weights change.

- Not in Hermes. Not in Claude Code. Not anywhere today.
- "Not retraining" just means "no fine-tuning."
- Everything here is *context engineering* — files written to disk and re-injected into a prompt.

Why local agents, now

Persistence

State survives the session without a vendor's storage model in the loop.

Privacy

The context layer is the sensitive part — your files, your clients.

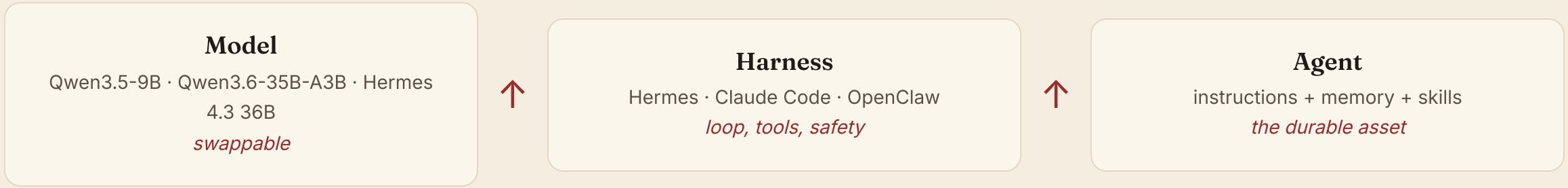
$0 marginal

The 400th cron run costs electricity. That's what makes always-on reasonable.

What actually changed in 2026

Not the models. Hermes (Feb 2026) made a memory-carrying local agent turnkey. The pieces existed; the bundling didn't.

Three layers people keep collapsing



Hermes is a persistent personal-agent harness. Claude Code is a coding harness that happens to persist a little.

Five primitives, and their Claude Code neighbor

Hermes primitive	What it is	Claude Code neighbor
Tools	fs, ripgrep, terminal, web, browser	Read/Edit/Bash/Grep
Integrations + MCP	messaging providers + any MCP server	MCP servers
Skills	`SKILL.md`, progressive disclosure	Skills — *same* standard
Cron	scheduled self-contained runs	`/loop`, cloud agents
Memory	`MEMORY.md` + `USER.md` + FTS5 log	`CLAUDE.md` + Auto Memory

The vocabulary maps ~1:1. What differs is what runs **by default and unattended**.

Where things live

Purpose	Hermes	Claude Code
Project instructions	`AGENTS.md` (repo root)	`CLAUDE.md` or the same `AGENTS.md`
Global personality	`SOUL.md` (`~/.hermes/`)	`~/.claude/CLAUDE.md`
Learned facts	`MEMORY.md` , `USER.md`	Auto Memory
Procedures	`~/.hermes/skills/*/SKILL.md`	`~/.claude/skills/*/SKILL.md`
Config	`config.yaml` , `.env`	`settings.json`

Picking a local model — ask in this order

1 · RAM
A hard wall, not a preference

→

2 · Task
Long-horizon tool use punishes small models

→

3 · Tool calls
The one that actually decides it

A model that writes beautiful prose and mangles a JSON tool call is *useless in a harness*.

The shortlist

Model	RAM	Where it fits
Qwen3.5-9B	8 GB+ at 64K · ~10 GB at 128K	Official Mac starter pick, quantized
Qwen3.6-35B-A3B	32 GB+	Daily driver, NVIDIA-endorsed default
Hermes 4.3 36B	32 GB+	Nous's own, tuned for tool calling

Hard requirement regardless: the model must serve ≥64K context.

System prompt + context files + tool defs + a real conversation does not fit in 8K.

The tool-call parser story

Every model family emits tool calls differently — XML-ish, JSON blocks, custom tokens.
The harness needs a **parser per family** to normalize them.

Nous ships a first-class `--tool-call-parser hermes` for vLLM.

> When a local model "doesn't work" with an agent, **the parser is the first suspect** — not the model's intelligence.

The landscape, one line each

Hermes

Persistent workflows, learns from work, one agent across CLI *and* messaging.

Claude Code + Ollama

Deep IDE integration, mature tooling, focused sessions.

OpenClaw

Lightest setup, pure CLI coding.

The rule: pick Hermes when the accumulated context is the point; pick a stateless coding agent when each session is disposable.

BLOCK 2 · HANDS-ON

Your first *Hermes agent*

Plumbing before personality.

INSTALL

Windows: PowerShell one-liner. Mac: also a Desktop installer.

`pip install hermes-agent` exists (Nous-published) but lags the installer — use the installer.

POINT IT AT A LOCAL MODEL

`-c 131072` + q4 KV ≈ 10 GB — on 8 GB use `-c 65536`.

Mac path: llama.cpp or MLX — not Ollama

- The official Mac guide points at **llama.cpp** (lower latency, `:8080`) or **omlx/MLX** (higher throughput, `:8000`) behind a Custom endpoint.
- Ollama works (`OLLAMA_API_KEY`) — it just isn't the doc's Mac path.
- **Pick one server and stick with it.** Mixing them mid-session produces unreproducible failures.

Claude Code contrast: you never pick a model *server*, only a model name. Here you own the whole inference stack.

Two context files, two scopes

`SOUL.md`

`~/.hermes/SOUL.md`

- Personality, values, boundaries, tone
- **Global and durable**
- Survives a model swap

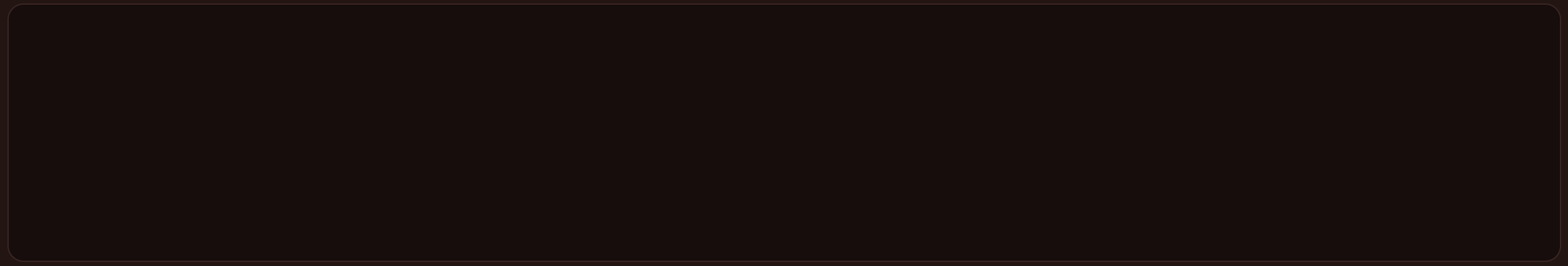
`AGENTS.md`

project root

- Architecture, conventions, hard rules
- **Per-project**, loaded from CWD at session start
- Subdirectory copies load lazily

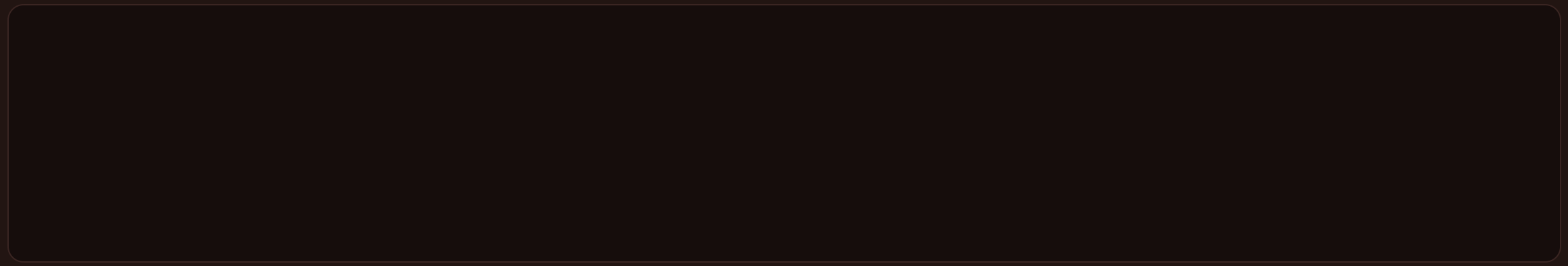
Bonus: Hermes auto-reads `.cursorrules` and `.cursor/rules/*.mdc`.

SOUL.MD — THE ANTI-SYCOPHANCY DEMO



Personality is a **context artifact, not a weights artifact.**

AGENTS.MD — THE PROJECT BRAIN



Context files and the prompt cache

Context files are part of the **stable system prompt** — which is what earns prompt-cache hits.

- Editing them mid-session **breaks the cache**. You pay a full re-read.
- Write them once, deliberately, at session start.
- Every character costs on *every turn*.

Turn on tools in blast-radius order

Rung 1 · Read-only
filesystem read, ripgrep → worst case: wasted tokens

Rung 2 · Draft-only
new scratch files; generate commands, *don't* execute → a junk file

Rung 3 · Reversible
edit files inside a git repo → `git checkout .`

Rung 4 · Irreversible
delete, terminal, send, spend, prod → you explain it to someone

Rung 2 is the underrated one: **read the command yourself before running it.**

Command approval is the safety net

Every command is checked against a curated dangerous-pattern list — recursive deletes, SQL drops, piping curl into a shell.

Four answers: **once / session / always / deny**

Default `mode: smart` auto-decides the clear cases; set `manual` to see the prompt.

> Choose **`session`**, not `always`. `always` is a permanent grant made at the moment you have the least evidence.

Same decision as Claude Code's permission modes. Identical failure mode.

Run one real task — and leave it uncut

- Watch tool calls land against your *local* model.
- Point at where the per-model parser normalizes the format.
- **Failures are the lesson.** Don't edit them out.

Prompting discipline

- **Be specific.** "Fix the `TypeError` in `api/handlers.py:47`" — not "fix the code."
- **Front-load context.** Paste the traceback. Paste the screenshot (`Ctrl+V`).
- **Let the agent use its tools.** "Find and fix the failing test" beats dictating each step.

Same instinct as Claude Code: dictate the goal, not the path.

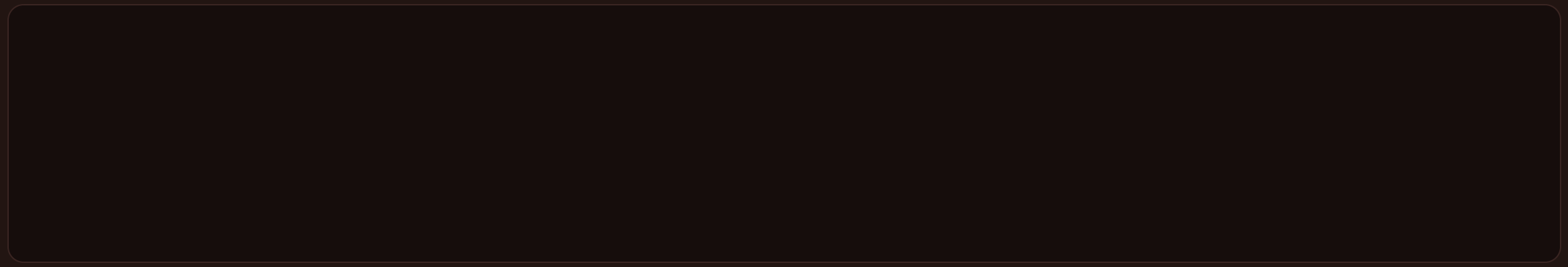
CLI power-use

Key / command	Does
`Alt+Enter` · `Ctrl+J` · `Shift+Enter`	Multi-line input
`Ctrl+C` once	Interrupt & redirect · twice in 2s = exit
`Ctrl+V`	Paste a clipboard image (vision)
`/` + `Tab`	Autocomplete commands and skills
`/verbose`	Cycles off → new → all → verbose
`hermes -c` · `hermes -r "title"`	Resume by history / by title

The 60-second eval habit

- You cannot tell whether a local model is good enough by vibes.
- "The answer felt right" is not a signal.
- The signal is: **did it call the right tool with the right arguments?**

25 ROWS OF REAL TASKS



Score two columns only: **right tool?** and **right arguments?**

What the eval set buys you

1

A go / no-go on a local model — below ~80% you'll blame the wrong layer

2

An objective comparison of two models — same 25 rows, two numbers

3

A regression check — adding tools *can* degrade tool selection

Re-run it after Block 3's skill and Block 4's MCP server.

BLOCK 3

Memory and skills *in action*

The part only Hermes turns on by default.

Three kinds of durable state

Kind	What it is	Where
Authored instructions	What you told it	`SOUL.md`, `AGENTS.md`
Learned facts	What it noticed and kept	`MEMORY.md`, `USER.md`
Learned procedures	How to do a repeated thing	`skills/*/SKILL.md`

Plus a fourth thing that's history, not state: an **SQLite/FTS5 log of every session**, searchable with `session_search`.

Teach the accuracy note, don't hide it

Nous markets a **"four-tier memory."** What the docs describe:

- `MEMORY.md` + `USER.md` + the FTS5 session log
- Honcho and Obsidian are **opt-in add-ons**, not built-in tiers
- `SOUL.md` is counted as a tier but is **config/identity — you wrote it, it didn't learn it**
- The Curator's **LLM-consolidation is OFF by default** (50–100 API calls a pass)

Presenting this straight buys you the whole room.

Memory is bounded, and frozen mid-session

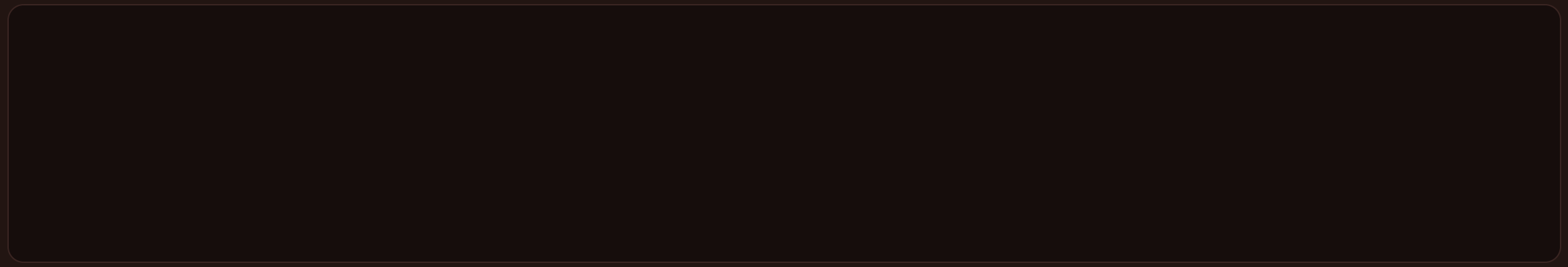
2,200
chars in MEMORY.md

1,375
chars in USER.md

next
session before a write reaches the system prompt

Small on purpose — these sit in the system prompt on *every* turn. Force a cleanup with *"clean up your memory."*

DEMO THIS LIVE



Not a bug. The write landed on disk; it reaches the system prompt next session.

Show it, or a student hits it alone at 11pm and concludes the feature is broken.

Session search — the genuinely distinct one

`session_search` runs FTS5 over **all** past raw conversations. Nothing loads until it's needed.

> This is one of two things a Claude Code power user can't replicate without building it. Claude Code's memory is curated notes — **not indexed transcripts.**

"Reference document → skill. Sticky note → memory."

— Hermes docs, on what goes where

Skills: procedural, and free until used

Same standard as Claude Code — agentskills.io, `SKILL.md`, three stages:

1 · Name + description
~100–200 tokens
always in context

→

2 · SKILL.md body
loads on trigger
keep under ~500 lines

→

3 · Reference files
read on demand
one at a time

In Hermes these are literal calls: `skills_list()` → `skill_view(name)` → `skill_view(name, file)`.

Promote to a skill when a task is 5+ steps *and* repeats.

Two authoring rules that fall straight out

- **Directives, not essays.** Layer 1 is an always-on cost on every model call.
- **If the workflow is a fixed deterministic sequence — write a script, not a skill.** Don't spend model tokens on work a bash file does perfectly.

The objection you will get

> *"Claude Code can also write files and update `CLAUDE.md`. So what?"*

They're right. No weights change in any of these tools.

The real question isn't *can* an agent persist state — they all can.
It's **how much of the loop is autonomous, always-on, and default.**

The self-improvement ladder

Rung 1
Can write a file if told. Proves nothing.

Rung 2
User-triggered persistence — `CLAUDE.md`, `AGENTS.md`. Authored, not curated.

Rung 3
Autonomously decides what to remember, unprompted.

Rung 4
+ unattended long-horizon curation; retrieval by *search*.

Rung 5
+ autonomous skill synthesis *and retirement* from usage stats.

Where each tool actually lands

- **Claude Code → rung 3.** Auto Memory decides what to keep. It has the full substrate for 4–5 — skills are first-class and portable — but a human or a hook drives skill creation, and nothing prunes old memory in the background.
- **Letta (MemGPT) →** arguably **ahead on rung 4.** Self-editing memory has been its architecture since 2023, plus "sleep-time" consolidation. No skill authoring.
- **Hermes →** the one documented running **3→5 wired together, on by default.**

Say it out loud: the differentiator is default integration, not intelligence.

Then re-run it as `/deploy-staging`

Five minutes of agent reasoning becomes **one slash command**, loaded instantly.

That contrast is the whole block. It lands better shown than described.

Then re-run the eval CSV. A new skill changes the always-loaded description budget and can shift tool selection. Prove it didn't regress.

The community skill layer

Skills you write for Claude Code are portable to Hermes and back. Same format, same progressive-disclosure contract.

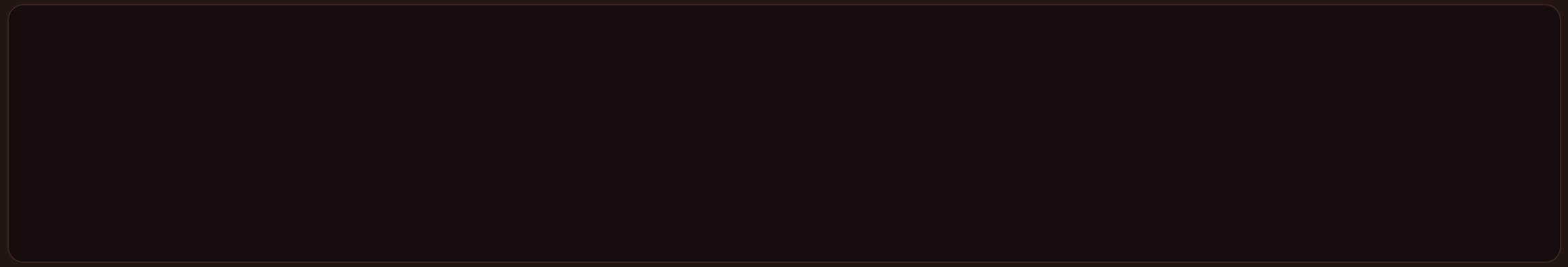
Your skill library is not harness lock-in. Consumer first, then producer.

BLOCK 4

MCP, deployment, *and reach*

Wire it low-blast-radius first. Prove it. Then widen.

~/.HERMES/CONFIG.YAML



Tools register as `mcp_<server>_<tool>` — useful when debugging a wrong pick.

`tools.include` is a security control

Not a tidiness feature. An MCP server hands the agent whatever it feels like exposing.

> An MCP server's tool **descriptions enter your model's context.** A malicious one is a prompt-injection surface. Read what you install.

OAuth and mTLS supported for remote servers. Parallel calls work.

Hermes as an MCP server

Exposes Hermes's **messaging bridge** — list, read and send conversations across your connected platforms. Claude Code can message your Telegram through Hermes.

These aren't competing harnesses. One can be the other's tool.

Then re-run the eval CSV. New tools, new chances to pick the wrong one.

Where it runs — 7 execution backends

Backend	Use it for
Local	Your machine, your files, fastest
Docker	Untrusted code, isolation, reproducible env
SSH	An always-on box you already own
Modal · Daytona · Vercel Sandbox	Serverless, hibernates, **~$0 idle**

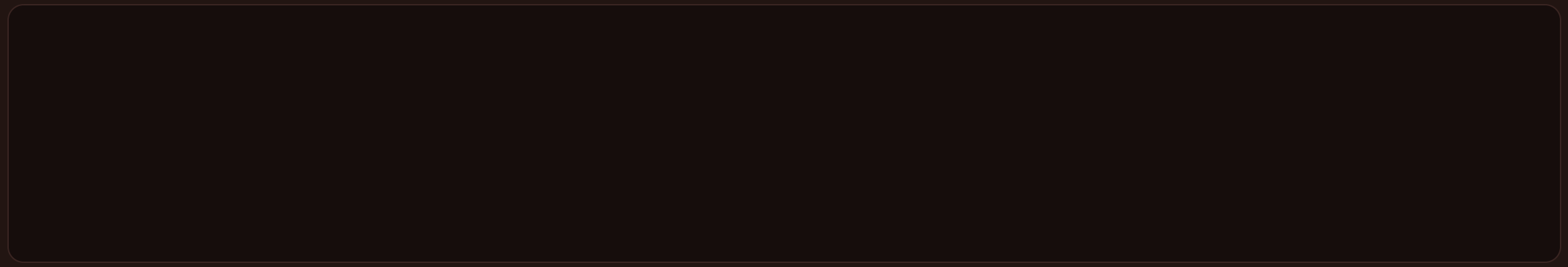
The brain — memory, skills, config — is a directory of files. The backend under it is swappable. **Tasks survive crashes and reboots.**

The gateway: one agent, everywhere you type

- **The same agent** on CLI, Telegram, Slack, Discord, Email — 30+ platforms
- Per-chat session state that **follows you across devices**
- Sessions never auto-reset unless you opt in via `session_reset`

The capability with no Claude Code equivalent: the agent that holds your memory is the one answering from the train.

OPTIONAL LIVE ADD-ON · WHATSAPP



Via Baileys — no Meta dev account. Unofficial client: WhatsApp can ban the number.

CRON — DURABILITY WITH A SCHEDULE

Flag	Does
`--script`	Python preflight; stdout → agent context
`--skill`	Attach a skill (stackable)
`--deliver`	telegram / discord / slack / origin / local
`[SILENT]`	Suppress the message on a no-op

The cron gotcha, and the ladder

Cron runs in a fresh, self-contained session. No history, no implicit CWD.

- "Check the repo" is meaningless. "Check `~/projects/api`" is not.
- `[SILENT]` is what makes monitors tolerable — a health check that pings 364 times to say "fine" gets muted by day three.
- **First version: read-only, `--deliver local`.** Watch it a day. *Then* let it write and message you.
- Always `/cron run <name>` once before trusting a schedule.

The prompt cache is the cost lever

A **stable system prompt** earns cache hits. Local model → latency. Hosted → money.

What breaks it: `/model` switches · provider fallbacks · credential rotation · editing `SOUL.md` / `AGENTS.md` mid-session.

> Prefer starting a **fresh session** on the other model over bouncing back and forth mid-conversation.

`/compress` · `/usage` · `hermes prompt-size` (offline) · `/insights` (30-day patterns)

Scale out when one session is the wrong shape

- `delegate_task` — parallel independent subagents. Same instinct as Claude Code's subagents: fan out work whose *results* you need but whose intermediate output you don't want in context.
- `execute_code` — one batch script beats twenty terminal round-trips. Each round-trip is a full model call; the script is one.

Security: containers skip the danger check

> **Docker, Singularity, Modal, Daytona, and Vercel Sandbox SKIP the dangerous-command check.** The container *is* the security boundary — by design.

The right setup for untrusted code. **The wrong setup if you assumed approval prompts were still firing.**

Never do this

That is a public shell on your machine.

Plus: secrets in `~/.hermes/.env` — never in `config.yaml`, never in `AGENTS.md`.

BLOCK 5

When to use *which*

Pick correctly. Defend the pick.

The matrix

Use...	When it's best
Hermes	Persistent workflows, learning-from-work, one agent across CLI *and* messaging
Claude Code + Ollama	Deep IDE integration, mature tooling, focused coding sessions
OpenClaw	Lightest setup, pure CLI coding

The rule: pick Hermes when the accumulated context is the point; pick a stateless coding agent when each session is disposable.

Wanting both is the correct answer — `hermes mcp serve` lets Claude Code reach your messaging through Hermes.

The honest verdict: true, but not unique

	Persistent memory	Unattended curation	Cross-session FTS	Agent-skills	Default?
Hermes	Yes	**Yes** (Curator ~7d)	**Yes**	**Yes**	**Automatic**
Claude Code	Yes	Partial — trims, no prune	No	Not unprompted	Memory yes, skills manual
OpenClaw	Yes	Unverified	Unverified	Partial (ClawHub)	Unverified
Letta	Yes	**Yes** — deepest	Yes	No	Architecture

"Agent writes durable state, reads it back, model swappable" is **Letta's 2023 founding idea.** Leave "unverified" unverified on the slide.

For the Claude Code power user specifically

You've hand-built most of this already. **Two things you don't get automatically:**

1

Full-text search across every past raw session

2

An unattended job that prunes and merges memory + skills on a schedule, unasked

Both buildable — a hook, an SQLite index, a cron job. Which *is* the point.

Answer the sharp question with the ladder

> *"Claude Code writes files too. So what's actually different?"*

1. Filesystem access → rung 1. Proves nothing.
2. `CLAUDE.md` / `AGENTS.md` → rung 2.
3. Claude Code's Auto Memory → rung 3.
4. Unattended curation + search retrieval → rung 4.
5. Skill synthesis with a retirement lifecycle → rung 5.

Hermes ships 3→5 wired together and turned on by default. Real, checkable, and smaller than the launch copy implies.

Where to go next

- Official docs — the canonical reference
- `NousResearch/hermes-agent` on GitHub (MIT)
- **awesome-hermes-agent** — the community list
- **Hermes self-evolution repo** — agent-improving-itself experiments
- agentskills.io — the skill standard, shared with Claude Code

Ship the artifacts: `SOUL.md` · `AGENTS.md` · `config.yaml` · cron commands · the eval CSV. Reproducibility is the #1 complaint about every comparable course.

ONE SLIDE TO REMEMBER

The model is a component. The context is the *asset*.

Build the context deliberately · wire new power at low blast radius first · measure with an eval set instead of a feeling.